

Introduccion a Ruby

Juan Ignacio Raggio

May 22, 2025

Contents

1	Diferencias con Java	3
1.1	Vamos a usar Ruby 3.4	3
2	Hello world	3
3	No existen los operadores	3
4	Constantes	3
5	Comillas simples " vs comillas dobles "	3
6	Strings: Metodos	4
7	Variables	4
7.1	Variables de clase	4
7.2	Variables de instancia	4
8	Metodos	4
8.1	Metodos de instancia	4
8.2	Metodos de clase + self.class vs ClassName	5
9	Herencia Ruby	5
9.1	Herencia de metodos privados	5
10	Metodos	5
10.1	Metodos privados	5
10.1.1	Organizacion de privados vs publicos	6
10.2	Metodos Protegidos	6
10.3	Metodos con argumentos variables	6

11 Poner el return si es en la ultima linea es de mal estilo	6
12 to_s (toString en Ruby)	6
13 Inspect	7
14 Clases abstractas en Ruby	7
14.1 Ejemplo de figuras	8
14.1.1 Si un metodo termina en signo de pregunta -> El metodo retorna un booleano	9
15 == vs equal?	9
15.1 equal?(other)	9
15.2 ==(other)	9
15.3 eql?(other)	9
15.4 ===(other)	9
15.5 Ejemplo	9
16 Operadores	10
16.1 TrueClass - FalseClass	11
16.2 Resolucion de constantes	11
17 Sintaxis	11
17.1 if	11
17.2 if y unless	11
17.3 case	12
17.4 Rangos	12
17.4.1 Pertenencia de rangos	13
18 Blocks - Bloques de codigo	13
18.1 Blocks con metavARIABLES	14
18.2 Metodos que reciben bloques	14
18.3 Loops	15
18.3.1 while	15
18.3.2 until	15
18.3.3 for	15
18.3.4 Loops como modificadores	15
19 Excepciones	16
19.1 Excepciones propias	16

20 Modulos	17
20.1 Modulo Kernel	17
20.2 Modulos: include	17

1 Diferencias con Java

- Es interpretado
- Tipado dinamico
- Todo es un objeto
- Garbage collector
- Manejo de excepciones
- Programacion funcional

1.1 Vamos a usar Ruby 3.4

2 Hello world

```
puts "Hello world"
```

3 No existen los operadores

- El <, >, == son todos metodos de instancia de cada objeto

4 Constantes

- Se escriben en mayusculas, si se cambia tira warning

5 Comillas simples " vs comillas dobles ""

- Las comillas simples son para tomar literal lo que esta dentro de las comillas
- Las comillas dobles son para tener **formato** -> Si no se quiere formato es ineficiente usar comillas dobles

```
name = 'juani'
saludo = "Hola #{name}"
```

6 Strings: Metodos

```
'1234'.size # 4
'1234ñ'.ascii_only:? # false
'abc'.capitalize
```

7 Variables

7.1 Variables de clase

- Comienzan con @@

7.2 Variables de instancia

- Comienzan con @
- No se pueden tener dos metodos con el mismo nombre (no existe el polimorfismo parametrico)
- Las variables de instancia son todas **privadas** si quiero permitir acceso hay que hacer un getter

```
class Date
  def initialize(day, mont, year)
    # variables de instancia
    @day = day
    @month = month
    @year = year
  end
end
```

8 Metodos

8.1 Metodos de instancia

- Comienzan con def
- Son todos publicos

```
# Dos formas de escribir lo mismo
def hello_world
  puts 'Hello world'
```

```
end
```

```
def simple_method = puts 'Hi!'
```

8.2 Metodos de clase + self.class vs ClassName

```
class Father
  def self.default_make = 'Father'

  def father_one = self.class.default_make
  def father_two = Father.default_make
end
```

9 Herencia Ruby

- Se usa el operador < que es equivalente al extends de Java

```
class Son < Father
  def self.default_make = 'Son'
end
```

9.1 Herencia de metodos privados

- Las clases hijas pueden acceder a los metodos privados de la clase padre

10 Metodos

10.1 Metodos privados

- Un metodo privado solo puede ser accedido por la propia instancia

```
class Father
  private def aux_method # mal estilo
    # private code
  end

  def some_method(other) = other.aux_method
end
```

```
# Falla
father = Father.new
father.aux_method

# Falla tambien
a = Father.new
b = Father.new
b.some_method(a) # Pues a no es this dentro de b entonces no tiene acceso
```

10.1.1 Organizacion de privados vs publicos

- Hay dos opciones estan en la diapo

10.2 Metodos Protegidos

- A diferencia de los metodos privados, otras clases si pueden acceder a los metodos protegidos
- Pero siguen siendo “privados” en cuanto al main u otra clase externa

10.3 Metodos con argumentos variables

- Un argumento que puede tener un conjunto de valores
- Solo se puede tener un argumento variable

```
def foo(a, b, *v, c)

end
```

11 Poner el return si es en la ultima linea es de mal estilo

12 to_s (toString en Ruby)

```
class Person
  def initialize(first_name, last_name)
    @first_name = first_name
    @last_name = last_name
  end
end
```

```

person = Person.new('John', 'Doe')
puts person

class Person
  def to_s
    "Person #{@first_name} #{@last_name}"
  end
end

puts person

```

13 Inspect

```

class Person
  def initialize(first_name, last_name)
    @first_name = first_name
    @last_name = last_name
  end
end

person = Person.new('John', 'Doe')

# En vez de hacer to_s usa inspect
p person

class Person
  def inspect
    "Person #{@first_name} #{@last_name}"
  end
end

p person

```

14 Clases abstractas en Ruby

- No existe palabra clave pero en caso de intentar inicializarla tiramos un error

```

class AbstractClass

```

```

def initialize
  raise 'Cannot instantiate an abstract class'
end

def init(some_var)
  @some_var = some_var
end

private :init
end

```

14.1 Ejemplo de figuras

- raise unless
- Instance of -> Considera la clase exacta, seria como un getClass de java
- is_a -> Tiene en cuenta herencia

```

class Point
  def initialize(x, y)
    @x = x
    @y = y
  end
end

```

```

class Square
  attr_accessor :top_left, :side
  def initialize(top_left, side)
    raise 'Invalid top left' unless top_left.is_a?(Point)
    raise 'Invalid side' unless side > 0 # Aca seria mejor usar un .positive?
    @top_left = top_left
    @side = side
  end
end

```

```

class Rectangle
  def initialize(top_left, height, width)
    raise 'Invalid top left' unless top_left.instance_of?(Point)
    raise 'Invalid height' if height <= 0
  end
end

```



```

        raise 'Invalid width' if not width.positive? # En vez de usar un if not es de mejor
        @top_left = top_left
        @height = height
        @width = width
    end
end

```

14.1.1 Si un metodo termina en signo de pregunta -> El metodo retorna un booleano

15 == vs equal?

15.1 equal?(other)

No sobrescribirlo

`a.equal?(b)` # Verdadero si a y b son el mismo objeto (misma direccion de memoria)

15.2 ==(other)

Sobrescribirlo

`a == b` # Es igual al `equal?(other)` por defecto, pero se recomienda sobrescribirlo

```

# En realidad eso esta haciendo
a.==?(b)

```

15.3 eql?(other)

Utilizado por colecciones que usan una tabla de hash Deberia llamar al `==`

15.4 ===(other)

“case equality”. Por defecto es lo mismo que `==`. Se sobrescribe para proveer semantica para sentencia **case**.

15.5 Ejemplo

- No usar if not, es mejor unless

```

class Person
  def initialize(id) = @id = id

```

```

def ==(other)
  return false unless other.is_a?(Person)
  # Esto me obliga a hacer un getter
  # Ademas ese getter tiene que ser protected
  @id == other.id
end

protected attr_reader :id

# Son equivalentes:

#protected def id = @id

#protected def id
#  @id
#end
end

```

16 Operadores

- Todo es un objeto en ruby, incluso los operadores
- Solo **nil** y **false** se evaluan como falso
- El metodo **&&** por defecto retorna el parametro recibido en el mensaje, salvo para la instancia **false** que devuelve **self**

```

true && false # => false
true || false # => true
1 > 3 || 2 < 5 # => true
(1 > 3 || 2 < 5).class # => TrueClass
4 && 5 # => 5
5 && 4 # => 4
5 || 4 # => 5
false && cualquier_cosa # => false
true || cualquier_cosa # => true
(nil && cualquier_cosa).nil? # => true

```

16.1 TrueClass - FalseClass

- **true** es la unica instancia de la clase **TrueClass** y representa el valor verdadero en expresiones booleanas
- Lo mismo para **false**

16.2 Resolucion de constantes

```
class
```

```
end
```

```
# Acceso a constantes
```

```
puts P0002::AUTHOR
```

17 Sintaxis

17.1 if

```
a = 1
if a == 0
  puts 'a is zero'
elsif a == 1
  puts 'a is one' # Imprime 'a is one'
elsif a >= 1
  puts 'a is greater than or equal to one'
else
  puts 'a is some other value'
end
```

17.2 if y unless

- Mal estilo vs buen estilo
- En general los metodos de ruby son cortos
- Es “mas facil” de leer

```
# Mal Estilo
if some_condition
  do_something
```

```

end

# Bien
do_something if some_condition

puts 'positivo' if a > 0      # bien
puts 'positivo' if a.positive? # mejor

puts 'non-zero' if not a == 0 # feo
puts 'non-zero' if a != 0     # ok
puts 'non-zero' unless a == 0 # bien
puts 'non-zero' unless a.zero? # espectacular

```

17.3 case

- Comparar un objeto contra multiples patrones. Los patrones son mat-
echeados con el metodo `===()`

```

case a
when 1, 2
  puts 'a is one or two'
when 3
  puts 'a is three'
else
  puts 'I don\'t know what a is'
end

```

- Tambien es valido como una alternativa if-elseif

```

case
when a == 1, a == 2
  puts 'a is one or two'
when a == 3
  puts 'a is three'
else
  puts 'I don\'t know what a is'
end

```

17.4 Rangos

Ojo porque el `*p*` es como el `puts` pero imprime el `inspect`, no tiene nada que ver con rangos

- Dos puntos es inclusivo
- Tres puntos es exclusivo (excluyendo al de la derecha)

```
p (1..5).class # Range.new(1, 5) => Range
p (1..5).max # => 5
p (1...5).max # Range.new(1, 5, true) => 4
p (1..5).min # => 1
p (2..5).min # => 2
p (1...1).min # => nil
p (...5).size # Beginless Ranges => Infinity
p (1...).size # Endless Ranges => Infinity

p (1..3).overlap?(2..5) # => true
p (1..3).overlap?(4..5) # => false
p (1..4) == (1..4) # => true
p (1..4).equal?(1..4) # => false
```

17.4.1 Pertenencia de rangos

- Se pueden hacer rangos de cualquier tipo de objeto
- Solo hay que sobrescribir successor
- El ===() evalua patrones

```
p (1..10) === 15 # => false
p (1..10) === 3.14159 # => true
p ('a'..'j') === 'z' # => false
p ('a'..'j') === 'c' # => true

p 5 == (1..10) # => false
p 5 === (1..10) # => false
p 5.equal?(1..10) # => false

p (1..10) === 5 # => true
p (1..10) == 5 # => false
p (1..10).equal?(5) # => false
```

18 Blocks - Bloques de código

- Un bloque esta formado por “pedazos” de códigos

- Tiene un nombre asociado
- Se encierra entre las sentencias **do end** o entre llaves
- Para invocarlo se usa la sentencia **yield**

Tambien es una alternativa a ciclos

```
5.times do # Entre do y end va el bloque de codigo
  puts 'Hello world'
end
```

```
# Es equivalente a
5.times { puts 'Hello world' }
```

18.1 Blocks con metavARIABLES

- Uso de metavARIABLES y de la keyword `it`

```
# Imprime los números del 0 al 8
9.times do |e|
  p e
end
```

```
# Imprime los impares del 0 al 8
9.times do |e|
  p e if e.odd?
end
```

```
# Si es una sola proposición
9.times { |e| p e if e.odd? }
```

```
# “it” ya es la primer metavARIABLE del bloque
9.times { p it if it.odd? }
```

18.2 Metodos que reciben bloques

El pasaje de parametros y por bloques van por lugares distintos

```
def test
  puts 'You are in the method'
```

```

    yield
    puts 'You are again back to the method'
    yield
end

# Invocamos al método y le “pasamos” un bloque
test { puts 'You are in the block' }

def test
  yield 5
  puts 'You are in the method test'
  yield 100
end

# Invocamos al método y le “pasamos” un bloque
test { puts "You are in the block #{it}" }
```

18.3 Loops

18.3.1 while

Mientras true

```

a = 1
while a < 10 [do]
  puts a
  a += 1
end
```

18.3.2 until

Mientras false Exactamente igual al while

18.3.3 for

Es de mal estilo y poco utilizado

18.3.4 Loops como modificadores

Es valido y se usa bastante en docuemntacion oficial de ruby

```

a = 0
puts a while (a += 1) < 10

a = 0
puts a until (a += 1) == 10

```

19 Excepciones

- Una excepcion es una instancia de una clase que extiende Exception
- Todos los **errores** extienden a StandardError
- Con el bloque begin/rescue se pueden capturar las excepciones
- Solo se deben capturar **errores**

```

begin
  c = a/b
rescue ZeroDivisionError => e
  puts e.message
rescue => e
  puts e.class
  puts e.message
end

```

19.1 Excepciones propias

- Deben extender a **StandardError**
- super llama al mismo metodo de la clase padre con todos los parametros que recibio

```

class Point
  def initialize(x, y)
    raise InvalidPointError unless x.is_a?(Numeric)
    # Esto es como llamar al constructor e inicializar con el parametro que esta al lado
    raise InvalidPointError, 'Invalid y point' unless y.is_a?(Numeric)
  end
end

class InvalidPointError < StandardError

```



```

def initialize(msg = 'Invalid Point')
  super
end
end

```

20 Modulos

- Brindan una forma de agrupar metodos, constantes, clases
- mixins: Agregar funcionalidad a clases
- Se pueden utilizar en las clases de varias formas:
 - **include** Es como un preprocesador de C
 - **extend**
 - **prepend**

20.1 Modulo Kernel

Ya es incluido por la clase Object por lo que sus metodos pueden ser invocados desde cualquier parte

```

module Kernel
  def self.Array(arg)
  def self...
    # Mas metodos
  end
end

```

20.2 Modulos: include

- Comportamientos extra -> Modulos (Es como las interfaces de java)

```

module A
  def a1 = 1
end

```

```

module B
  def b1 = 3
  def a1 = 3
end

```

```
class Sample
  include A
  include B

  def s1 = 5
end

samp = Sample.new
puts samp.a1 # 3 (pues se incluye despues el B entonces se sobrescribe)
puts samp.b1 # 3
puts samp.s1 # 5
```